Développement web Front-End

Par: Kevin Noel Fonkoua Tatang (CTO EH2S Sarl, CEO LABO ACADEMY)

Syllabus

1. Informations générales

Rubrique	Détail
Intitulé	Développement Web Front-End
Niveau	Licence 2 (L2) - Informatique / Génie Logiciel
Volume horaire	24 h (8 séances de 3 h)
Organisation	Environ 30 % cours et démonstrations, 70 % travaux pratiques sur machine
Prérequis	Notions d'algorithmique (variables, conditions, boucles), utilisation d'un ordinateur et d'un navigateur
Insistance pédagogique	JavaScript (manipulation du DOM) et validation des formulaires : 18 h sur 24
Outils	VS Code (extension Live Server), Google Chrome / Firefox (DevTools), Git et GitHub

2. Présentation du cours

Ce cours donne aux étudiants les compétences nécessaires pour concevoir des interfaces web statiques et dynamiques. Après une mise à niveau rapide sur HTML5 et CSS3, le cours se concentre sur **JavaScript**, la **manipulation du DOM**, la **gestion des événements** et surtout la **validation des formulaires côté client**, une compétence centrale pour tout développeur front-end.

Un **fil rouge** accompagne toutes les séances : un site de type « inscription à un service » dont le formulaire est progressivement enrichi, jusqu'à une validation complète, accessible et ergonomique.

3. Objectifs d'apprentissage

À la fin du cours, l'étudiant sera capable de :

- Structurer une page web avec un HTML5 sémantique et accessible ;
- Mettre en forme une page responsive avec CSS3 (Flexbox, Grid, media queries) ;
- Écrire du JavaScript moderne (ES6+) : variables, fonctions, tableaux, objets, modules simples ;
- Sélectionner, créer, modifier et supprimer des éléments du DOM ;
- Gérer les événements (clic, saisie, soumission, propagation, délégation) ;

- Concevoir et implémenter la validation d'un formulaire (HTML5, Constraint Validation API, expressions régulières, règles personnalisées) ;
- Afficher des retours utilisateur clairs et accessibles (messages d'erreur, états visuels, ARIA) ;
- Déboguer avec les outils de développement du navigateur ;
- Versionner et livrer un mini-projet avec Git et GitHub.

4. Répartition du volume horaire

Séance	Thème	Durée
1	HTML5 et structure d'une page	3 h
2	CSS3, mise en page et responsive	3 h
3	JavaScript : les fondamentaux	3 h
4	JavaScript : tableaux, objets, ES6+	3 h
5	Le DOM : sélection et manipulation	3 h
6	Événements et DOM avancé	3 h
7	Validation de formulaires (1) : HTML5 et JavaScript	3 h
8	Validation de formulaires (2) : regex, UX, accessibilité et projet	3 h
	Total	**24 h**

5. Programme détaillé

Séance 1 : HTML5 et structure d'une page (3 h)

Objectifs : comprendre le fonctionnement du web et produire une page HTML5 propre.

- Fonctionnement du web : client/serveur, HTTP, navigateur, rôle du front-end.
- Mise en place de l'environnement : VS Code, Live Server, DevTools, dépôt Git.
- Structure d'un document HTML5 : `doctype`, `head`, `body`, métadonnées, encodage.
- Balises sémantiques : `header`, `nav`, `main`, `section`, `article`, `aside`, `footer`.
- Texte, liens, images, listes, tableaux.
- Introduction aux formulaires : `form`, `label`, `input` (types `text`, `email`, `password`, `number`, `date`, `radio`, `checkbox`), `select`, `textarea`, `button`.
- Attributs essentiels : `id`, `name`, `class`, `placeholder`, `for`.

TP : construire la page d'inscription du fil rouge (structure HTML uniquement).

Séance 2 : CSS3, mise en page et responsive (3 h)

Objectifs : styliser une page et la rendre adaptable à tous les écrans.

- Intégration du CSS (fichier externe), sélecteurs, spécificité, cascade.
- Modèle de boîte (`margin`, `border`, `padding`), unités (`px`, `rem`, `%`, `vw`).
- Couleurs, typographie, arrière-plans.
- Mise en page avec **Flexbox** et **Grid**.
- Responsive design : approche mobile-first, media queries.

- Pseudo-classes utiles pour les formulaires : `:focus`, `:hover`, `:valid`, `:invalid`, `:disabled`.
- Variables CSS et bonnes pratiques de nommage (BEM).

TP : mettre en forme la page d'inscription, la rendre responsive et préparer les classes CSS d'état (`.error`, `.success`).

Séance 3 : JavaScript, les fondamentaux (3 h)

Objectifs : maîtriser la syntaxe et la logique de base du langage.

- Rôle de JavaScript, intégration dans une page (`script`, `defer`), console du navigateur.
- Variables : `let`, `const`, portée ; types primitifs, conversions, `typeof`.
- Opérateurs, comparaison stricte (`===`), valeurs *truthy* et *falsy*.
- Structures conditionnelles : `if/else`, `switch`, opérateur ternaire.
- Boucles : `for`, `while`, `do…while`, `for…of`.
- Fonctions : déclaration, expression, fonctions fléchées, paramètres, valeur de retour.
- Chaînes de caractères : méthodes courantes (`trim`, `includes`, `slice`, `toLowerCase`), template literals.
- Débogage : `console.log`, points d'arrêt, lecture des erreurs.

TP : série d'exercices (calculatrice, vérification de parité, validation simple d'un âge, mini-fonctions utilitaires pour contrôler du texte).

Séance 4 : JavaScript, tableaux, objets et ES6+ (3 h)

Objectifs : manipuler des collections de données et écrire un code moderne.

- Tableaux : création, parcours, `push`, `pop`, `splice`, `find`, `includes`.
- Méthodes fonctionnelles : `map`, `filter`, `reduce`, `some`, `every`, `forEach`.
- Objets : propriétés, méthodes, accès, itération (`Object.keys`, `Object.entries`).
- Déstructuration, opérateur de décomposition (`...`), paramètres par défaut.
- JSON : `JSON.stringify`, `JSON.parse`.
- Gestion des erreurs : `try/catch`, `throw`.
- Notions de modules ES (`import` / `export`) et organisation du code.
- Stockage local : `localStorage` (lecture, écriture, suppression).

TP : gérer une liste d'utilisateurs (tableau d'objets), filtrer, trier, sauvegarder dans `localStorage`. Préparer un objet de règles de validation pour le fil rouge.

Séance 5 : Le DOM, sélection et manipulation (3 h)

Objectifs : comprendre l'arbre DOM et agir dessus par le code.

- Qu'est-ce que le DOM ? Arbre de nœuds, objets `document`, `element`, `node`.
- Sélection : `getElementById`, `querySelector`, `querySelectorAll`, collections et `NodeList`.

- Lecture et modification du contenu : `textContent`, `innerHTML` (risques de sécurité XSS), `value`.
- Modification des attributs : `getAttribute`, `setAttribute`, `removeAttribute`, propriétés (`disabled`, `checked`).
- Gestion des styles et des classes : `style`, `classList.add/remove/toggle/contains`.
- Création et insertion d'éléments : `createElement`, `append`, `prepend`, `insertBefore`, `remove`, `cloneNode`.
- Navigation dans l'arbre : `parentElement`, `children`, `nextElementSibling`.
- Attributs `data-*` et `dataset`.

TP : générer dynamiquement une liste de produits ou d'utilisateurs à partir d'un tableau d'objets, afficher/masquer des sections, remplir dynamiquement une liste déroulante (pays, villes).

Séance 6 : Événements et DOM avancé (3 h)

Objectifs : rendre une page interactive en réagissant aux actions de l'utilisateur.

- Modèle d'événements : `addEventListener`, `removeEventListener`, objet `event`.
- Événements courants : `click`, `dblclick`, `input`, `change`, `focus`, `blur`, `keydown`, `keyup`, `submit`, `DOMContentLoaded`.
- `event.target`, `event.currentTarget`, `preventDefault`, `stopPropagation`.
- Propagation : phases de capture et de bulle ; **délégation d'événements**.
- Manipulation de formulaires en JavaScript : `FormData`, `form.elements`, lecture des champs, `reset`.
- Temporisation : `setTimeout`, `setInterval`, notion de *debounce*.
- Introduction à l'asynchrone : `fetch`, promesses, `async/await` (aperçu avec une API publique).

TP : application « liste de tâches » complète (ajout, suppression, marquage, filtrage, persistance dans `localStorage`). Lecture des valeurs du formulaire d'inscription et affichage d'un récapitulatif.

Séance 7 : Validation de formulaires (1) : HTML5 et JavaScript (3 h)

Objectifs : valider les saisies avec les mécanismes natifs puis en JavaScript.

- Pourquoi valider ? Validation côté client vs côté serveur (la validation client ne remplace jamais le serveur) ; sécurité et expérience utilisateur.
- Validation native HTML5 : `required`, `minlength`, `maxlength`, `min`, `max`, `pattern`, `type="email"`, `type="url"`, `novalidate`.
- **Constraint Validation API** : `checkValidity()`, `reportValidity()`, `validity` (`valueMissing`, `typeMismatch`, `patternMismatch`, `tooShort`, `rangeOverflow`…), `validationMessage`, `setCustomValidity()`.
- Intercepter la soumission : événement `submit` et `preventDefault`.
- Validation par champ : événements `input`, `blur` et `change`.
- Écrire des fonctions de validation réutilisables (nom, e-mail, âge, téléphone, mot de passe).
- Afficher les erreurs dans le DOM : création d'un élément message sous chaque champ, ajout/suppression de classes `.error` / `.success`.

TP : implémenter la validation du formulaire d'inscription champ par champ, avec messages d'erreur personnalisés affichés dans le DOM.

Séance 8 : Validation de formulaires (2) : regex, UX, accessibilité et projet (3 h)

Objectifs : concevoir une validation robuste, ergonomique et accessible, puis finaliser le projet.

- **Expressions régulières** : syntaxe de base (classes, quantificateurs, ancres, groupes), `test()`, `match()`, `replace()`. Cas pratiques : e-mail, numéro de téléphone camerounais, mot de passe, code postal, matricule.
- Règles croisées : confirmation du mot de passe, cohérence de dates (naissance, début/fin), champs conditionnels.
- Indicateur de force du mot de passe, compteur de caractères, afficher/masquer le mot de passe.
- Validation en temps réel et bonnes pratiques d'ergonomie : quand et comment afficher une erreur, messages clairs, focus sur le premier champ invalide.
- Accessibilité : `label` associé, `aria-invalid`, `aria-describedby`, `role="alert"`, navigation au clavier.
- Sécurité côté client : échappement des saisies, éviter `innerHTML` avec des données utilisateur.
- Récapitulatif de fin de formulaire et simulation d'envoi (`fetch` vers une API de test).
- **Finalisation du projet** : revue de code, push sur GitHub, présentation courte.

TP / Projet : livraison du formulaire d'inscription complet (voir section 7).

6. Méthodes pédagogiques

- Cours interactif avec démonstrations en direct (*live coding*).
- Travaux pratiques sur machine à chaque séance, avec exercices progressifs.
- Fil rouge unique pour montrer l'évolution d'un même projet.
- Revue de code collective et correction des erreurs fréquentes.
- Travail en binôme pour le projet, avec suivi via Git et GitHub.
- Utilisation systématique des DevTools pour apprendre à diagnostiquer.

7. Projet fil rouge : « Formulaire d'inscription intelligent »

Chaque étudiant ou binôme réalise une page d'inscription (ou de demande de service) comportant au minimum :

- au moins 8 champs de types variés (texte, e-mail, mot de passe et confirmation, date de naissance, téléphone, liste déroulante, boutons radio, cases à cocher conditions d'utilisation) ;
- une mise en page responsive en CSS ;
- une validation en temps réel (`input` / `blur`) et à la soumission, avec messages d'erreur personnalisés ;
- au moins trois règles par expressions régulières et deux règles croisées ;
- un indicateur de force du mot de passe ;
- une interface accessible (`label`, ARIA, navigation clavier) ;

- un récapitulatif dynamique affiché dans le DOM après validation, avec sauvegarde dans `localStorage` ;
- un dépôt GitHub propre (commits réguliers, fichier README).

8. Modalités d'évaluation

Composante	Pondération	Contenu
Contrôle continu	30 %	TP notés et quiz courts (séances 3 à 6)
Projet fil rouge	30 %	Qualité du code, validation, ergonomie, accessibilité, dépôt Git
Examen pratique final	40 %	Épreuve sur machine de 2 h : manipulation du DOM, événements et validation de formulaire

Critères principaux : exactitude fonctionnelle, propreté et lisibilité du code, respect des bonnes pratiques, qualité des retours utilisateur, autonomie.

9. Compétences visées (référentiel)

Compétence	Niveau attendu
HTML5 sémantique et formulaires	Maîtrise
CSS3 et responsive	Application
JavaScript (syntaxe, ES6+)	Maîtrise
Manipulation du DOM	Maîtrise approfondie
Gestion des événements	Maîtrise approfondie
Validation de formulaires	Maîtrise approfondie
Débogage et outils du navigateur	Application
Git et GitHub	Initiation

10. Ressources et bibliographie

- MDN Web Docs (Mozilla) : référence HTML, CSS, JavaScript et DOM (developer.mozilla.org).
- javascript.info : *The Modern JavaScript Tutorial*.
- Marijn Haverbeke, *Eloquent JavaScript* (disponible en ligne gratuitement).
- W3C : spécifications HTML et recommandations WCAG pour l'accessibilité.
- regex101.com : test et explication des expressions régulières.
- Documentation Git et GitHub pour le travail collaboratif.

11. Règles de fonctionnement

- Présence et ponctualité obligatoires pour les séances de TP.
- Ordinateur personnel recommandé, avec VS Code et un navigateur récent installés avant la séance 1.
- Plagiat de code interdit : tout code externe doit être cité et compris.
- Les rendus se font par dépôt GitHub, dans les délais fixés en début de cours.

INITIATION AU HTML 5 + CSS 3

Présentation du projet

Dans ce cours, nous construisons **étape par étape** la page d'accueil de **Labo Academy**, un petit site de formation. Chaque notion HTML5 est d'abord expliquée, puis **appliquée tout de suite** dans le projet : à la fin, vous aurez une page complète avec en-tête, bannière, formations, formulaire d'inscription et pied de page.

Rubrique	Détail
Durée	3 h (séance 1 du cours de développement web front-end)
Niveau	Licence 2 (GID 2)
Résultat attendu	Une page `index.html` complète, valide et bien structurée (sans mise en forme : le CSS3 vient ensuite)
Outils	VS Code, extension Live Server, navigateur (Chrome ou Firefox)

Plan du cours

Étape	Notion	Application dans le projet
0	Préparer le projet	Dossier et fichier `index.html`
1	Le squelette d'une page HTML5	Page vide valide avec titre
2	Titres, paragraphes et texte	Textes de présentation
3	Les listes (`ul`, `ol`)	Avantages et étapes d'inscription
4	Liens et images (`a`, `img`)	Logo et menu
5	`div`, `span`, `class` et `id`	Regrouper et identifier les blocs
6	Structure d'une page web	`header`, bannière, `section`, `article`, `footer`
7	Le formulaire	Formulaire d'inscription
8	Vérification finale	Validation W3C et checklist

Principe : tapez le code vous-même plutôt que de le copier. Après chaque étape, enregistrez et regardez le résultat dans le navigateur.

Étape 0 : Préparer le projet

Objectif : avoir un dossier propre dès le départ.

Créez l'arborescence suivante :

```
labo-academy/
├── index.html
├── img/
└── (css/ et js/ viendront plus tard)
```

Ouvrez le dossier dans VS Code, créez `index.html`, puis lancez **Live Server** (clic droit sur le fichier, « Open with Live Server »). La page se rafraîchira automatiquement à chaque sauvegarde.

Étape 1 : Le squelette d'une page HTML5

Objectif : écrire la structure minimale que toute page web doit avoir.

Ce qu'est le HTML : HyperText Markup Language, un langage de **balisage**. On entoure le contenu avec des balises qui disent au navigateur *ce que c'est* (un titre, un paragraphe, un lien…). Ce n'est pas un langage de programmation : il décrit la structure, pas la logique.

Anatomie d'une balise

```
<a href="https://exemple.com" target="_blank">Mon lien</a>
```

- `<a>` est la **balise ouvrante**, `</a>` la **balise fermante** ;
- `href` et `target` sont des **attributs** (ils précisent le comportement de la balise) ;
- `Mon lien` est le **contenu** ;
- certaines balises sont **orphelines** (sans fermeture) : `<br>`, `<hr>`, `<img>`, `<input>`.

Le squelette : dans `index.html`, écrivez :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Labo Academy : formations en développement web,
machine learning et génie logiciel.">
  <title>Labo Academy – Accueil</title>
</head>
<body>
  <!-- Le contenu visible de la page ira ici -->
</body>
</html>
```

Explications ligne par ligne

- `<!DOCTYPE html>` : indique au navigateur qu'il s'agit d'un document HTML5. Toujours en première ligne.
- `<html lang="fr">` : racine du document ; `lang` précise la langue (accessibilité et référencement).
- `<head>` : informations **invisibles** sur la page (titre de l'onglet, encodage, description, feuilles de style).
- `<meta charset="UTF-8">` : affiche correctement les accents et les caractères spéciaux.
- `<meta name="viewport" …>` : nécessaire pour que la page s'affiche correctement sur téléphone.
- `<title>` : texte affiché dans l'onglet du navigateur et dans les résultats de recherche.
- `<body>` : tout le contenu **visible** de la page.

- `<!-- … -->` : commentaire, ignoré par le navigateur.

Bonnes pratiques

- Indentez les balises imbriquées (2 espaces) : le code reste lisible.
- Fermez toujours les balises dans l'ordre inverse de leur ouverture : `<p><strong>…</strong></p>`, jamais `<p><strong>…</p></strong>`.
- Écrivez les noms de balises et d'attributs en minuscules.

Étape 2 : Titres, paragraphes et mise en valeur du texte

Objectif : structurer le texte de la page.

Les titres `h1` à `h6` : six niveaux, du plus important au moins important.

- Un **seul `h1`** par page : le titre principal.
- Ensuite `h2` pour les grandes parties, `h3` pour les sous-parties, sans sauter de niveau.
- Ne choisissez pas un titre pour sa taille (le CSS s'en occupe) mais pour son **niveau dans la hiérarchie**.

Les paragraphes `p` : un bloc de texte. Le navigateur ajoute automatiquement un espace entre deux paragraphes.

Autres balises de texte

Balise	Rôle	Exemple
` `	Retour à la ligne (balise orpheline)	`Douala Cameroun`
`<hr>`	Ligne de séparation thématique	entre deux parties
`<strong>`	Importance forte (gras)	`<strong>Gratuit</strong>`
`<em>`	Emphase (italique)	`<em>très</em> pratique`
`<small>`	Texte secondaire (mentions légales)	`<small>© 2026</small>`

Application au projet : dans le `<body>`, ajoutez provisoirement :

```
<h1>Bienvenue à Labo Academy</h1>
<p>Apprenez le <strong>développement web</strong>, le <em>machine learning</em> et le génie logiciel avec des projets concrets.</p>

<h2>Nos formations</h2>
<h3>Développement Web</h3>
<p>HTML5, CSS3 et JavaScript pour créer des sites modernes.</p>

<h2>Nous contacter</h2>
<p>
  Labo Academy<br>
  Douala, Cameroun<br>
  Ouvert du lundi au vendredi
</p>
```

À retenir

- `<br>` sert à couper une ligne **dans** un texte (une adresse, un poème). Ne l'utilisez **jamais** pour créer de l'espace entre deux blocs : c'est le rôle du CSS.
- Plusieurs espaces ou retours à la ligne dans le code sont réduits à un seul espace à l'affichage.

Étape 3 : Les listes

Objectif : présenter des éléments sous forme de liste.

Balise	Signification	Quand l'utiliser
`<ul>`	*Unordered list* : liste à puces	l'ordre n'a pas d'importance
`<ol>`	*Ordered list* : liste numérotée	l'ordre compte (étapes)
`<li>`	*List item* : un élément de la liste	toujours à l'intérieur de `ul` ou `ol`

Application au projet : remplacez le paragraphe de la formation Développement Web et ajoutez les étapes d'inscription :

```
<h3>Pourquoi choisir Labo Academy ?</h3>
<ul>
  <li>Des projets concrets à chaque cours</li>
  <li>Des enseignants en activité</li>
  <li>Un suivi personnalisé</li>
</ul>

<h3>Comment s'inscrire ?</h3>
<ol>
  <li>Remplir le formulaire en ligne</li>
  <li>Valider son adresse e-mail</li>
  <li>Choisir sa filière</li>
</ol>
```

Listes imbriquées : une liste peut contenir une autre liste, **à l'intérieur d'un `li`** :

```
<ul>
  <li>Front-end
    <ul>
      <li>HTML5</li>
      <li>CSS3</li>
      <li>JavaScript</li>
    </ul>
  </li>
  <li>Back-end</li>
</ul>
```

À retenir : les seuls enfants directs de `ul` et `ol` sont des `li`. Un menu de navigation est aussi, très souvent, une liste de liens : nous le ferons à l'étape suivante.

Étape 4 : Les liens et les images

Objectif : relier les pages entre elles et illustrer le contenu.

La balise `a` (ancre)

Sa force est l'attribut `href` (*hypertext reference*), qui contient la destination.

Type de lien	Exemple	Effet
Vers une autre page du site	`<a href="formations.html">Formations</a>`	ouvre la page locale
Vers un site externe	`<a href="https://www.w3.org" target="_blank" rel="noopener">W3C</a>`	ouvre dans un nouvel onglet
Vers une section de la même page	`<a href="#inscription">S'inscrire</a>`	fait défiler jusqu'à l'élément dont l'`id` est `inscription`
Vers une adresse e-mail	`<a href="mailto:contact@laboacademy.cm">Écrire</a>`	ouvre le client e-mail
Vers un numéro de téléphone	`<a href="tel:+237600000000">Appeler</a>`	lance l'appel sur mobile

- `target="_blank"` ouvre le lien dans un nouvel onglet ; ajoutez toujours `rel="noopener"` avec, pour la sécurité.
- Un texte de lien doit être explicite : préférez « Voir les formations » à « Cliquez ici ».

La balise `img`

```
<img src="img/logo.png" alt="Logo de Labo Academy" width="120">
```

- `src` : chemin de l'image (relatif au fichier HTML) ;
- `alt` : **texte alternatif**, obligatoire. Il est lu par les lecteurs d'écran et affiché si l'image ne charge pas ;
- `width` et `height` : réservent la place pour éviter que la page « saute » au chargement.

> *Si vous n'avez pas encore de logo, placez une image quelconque nommée `Logo.png` dans `img/` : l'important est de comprendre le chemin.*

Chemins relatifs et absolus

- `img/logo.png` : depuis le dossier du fichier HTML ;
- `../img/logo.png` : on remonte d'un dossier ;
- `https://…/logo.png` : adresse absolue (autre site).

Application au projet : créez le menu de navigation (provisoirement en haut du `<body>`) :

```
<nav>
  <ul>
    <li><a href="#accueil">Accueil</a></li>
    <li><a href="#formations">Formations</a></li>
    <li><a href="#inscription">Inscription</a></li>
    <li><a href="#contact">Contact</a></li>
```

```
  </ul>
</nav>
```

`<nav>` est une balise **sémantique** qui indique « ceci est un bloc de navigation ». Les liens commençant par `#` ne marcheront qu'après l'étape 6, quand nous aurons donné des `id` aux sections.

Étape 5 : Importance de `div`, `span`, `class` et `id`

Objectif : regrouper et identifier les éléments, pour pouvoir les cibler plus tard avec le CSS et le JavaScript.

`div` et `span` : des conteneurs génériques

Balise	Type	Usage
`<div>`	**Bloc** : occupe toute la largeur, passe à la ligne	regrouper plusieurs éléments (une carte, un logo + du texte)
`<span>`	**En ligne** : occupe seulement la place de son contenu	isoler un mot ou une partie d'une phrase

Ils n'ont **aucun sens** par eux-mêmes et aucun style particulier : ce sont des boîtes vides destinées à être regroupées et stylisées.

```
<div class="carte">
  <h3>Machine Learning</h3>
  <p>Prix : <span class="prix">50 000 FCFA</span></p>
</div>
```

`class` et `id` : donner une identité aux balises

Attribut	Unicité	Rôle	Exemple
`class`	**Réutilisable** : plusieurs éléments peuvent avoir la même classe ; un élément peut en avoir plusieurs	appliquer un même style à un groupe	`class="carte promo"`
`id`	**Unique** dans la page	identifier un élément précis (ancre, JavaScript)	`id="inscription"`

Pourquoi c'est indispensable ? Le CSS et le JavaScript ne « voient » pas le sens de vos balises : ils s'accrochent à des noms. Plus tard, nous écrirons par exemple :

- en CSS : `.carte { border: 1px solid grey; }` pour styliser toutes les cartes ;
- en JavaScript : `document.getElementById("form-inscription")` pour récupérer le formulaire.

Règles de nommage

- minuscules, sans accents ni espaces, mots séparés par un tiret : `form-inscription`, `carte-formation` ;
- nommer selon le **rôle** (`menu`, `banniere`) et non selon l'apparence (`rouge`, `gros-texte`) ;
- ne jamais utiliser le même `id` deux fois dans une page.

`div` ou balise sémantique ?

Utilisez une balise sémantique (`header`, `nav`, `section`, `article`, `footer`) **quand elle existe** pour votre besoin ; réservez `div` aux regroupements sans sens particulier (par exemple, regrouper un logo et du texte, ou envelopper une carte pour la mise en page).

Étape 6 : La structure d'une page web

Objectif : organiser la page en grandes zones, comme dans la plupart des sites.

Vue d'ensemble

Deux schémas résument l'organisation d'une page web : le premier donne le **vocabulaire** (les balises sémantiques), le second montre leur **agencement** sur une vraie page.

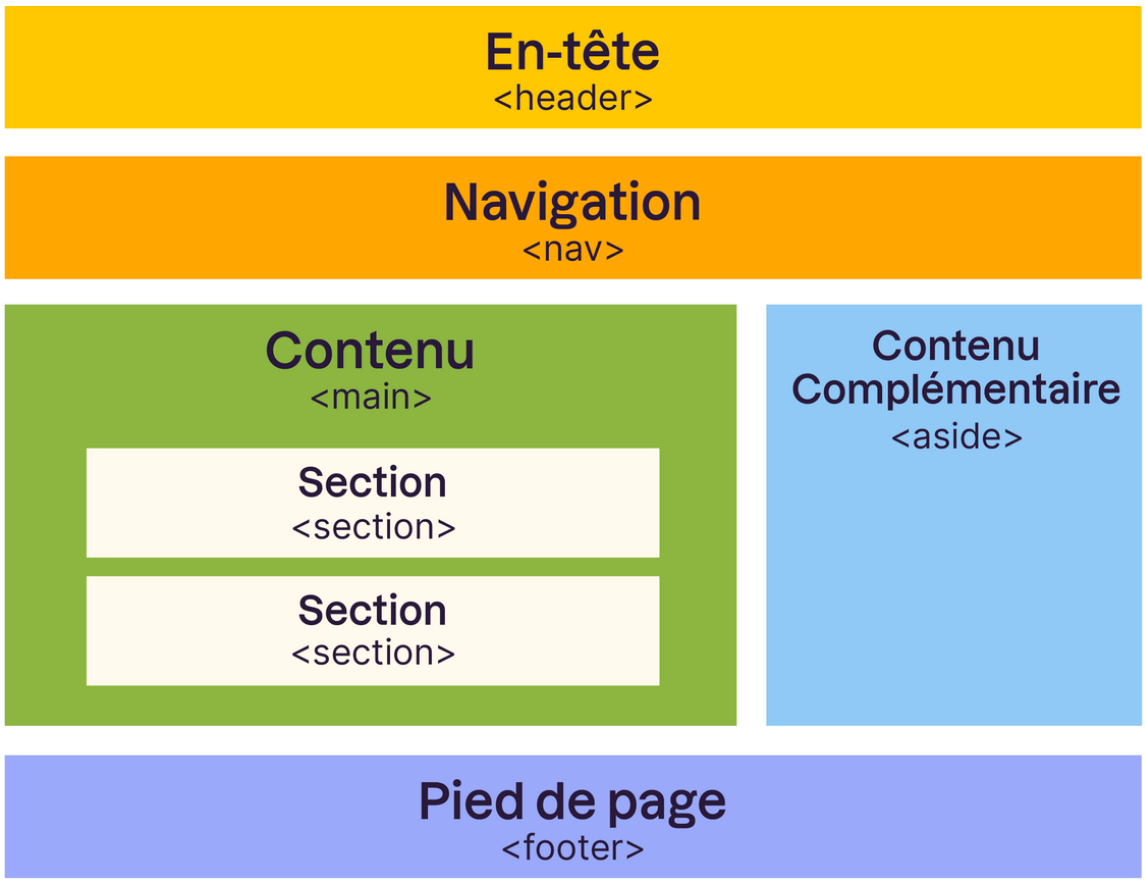


Figure 1 : les balises sémantiques HTML5 et leur emplacement dans la page.



Figure 2 : maquette d'un site. Le logo et le menu forment l'en-tête, la sidebar est un `aside`, la deuxième section regroupe trois `article`.

Structure de notre projet :

- `header` : `div.logo` + `nav.menu` ;
- `div.banniere` : titre d'accroche et bouton ;
- `main` : `section#formations` (trois `article`), `section#apropos`, `section#inscription` ;
- `footer#contact`.

Notre page suit ce modèle, avec une bannière en plus et sans `aside`, pour rester simple.

Rôle de chaque zone

Balise	Rôle
`<header>`	En-tête de la page (ou d'une section) : logo, menu.
`<nav>`	Bloc de liens de navigation principaux.

Balise	Rôle
`<div class="banniere">`	Zone d'accroche sous l'en-tête : grand titre, phrase forte, bouton d'action.
`<main>`	Contenu **principal** unique de la page.
`<section>`	Grande partie thématique, **avec un titre** (`h2`).
`<article>`	Contenu **autonome** qui pourrait vivre seul (carte de formation, billet de blog, actualité).
`<aside>`	Contenu secondaire lié (encadré, publicité).
`<footer>`	Pied de page : mentions légales, contact, liens utiles.

Section ou article ? Si le bloc a un sens isolé (une carte de formation compréhensible seule), c'est un `article`. S'il regroupe plusieurs éléments liés autour d'un thème (toutes les formations), c'est une `section`.

Application : assembler la page

Supprimez le contenu provisoire de l'étape 2 à 4 et remplacez le `<body>` par :

```
<body>

  <header class="entete" id="accueil">
    <div class="logo">
      <img src="img/logo.png" alt="Logo de Labo Academy" width="60">
      <span>Labo Academy</span>
    </div>
    <nav class="menu" aria-label="Navigation principale">
      <ul>
        <li><a href="#accueil">Accueil</a></li>
        <li><a href="#formations">Formations</a></li>
        <li><a href="#inscription">Inscription</a></li>
        <li><a href="#contact">Contact</a></li>
      </ul>
    </nav>
  </header>

  <div class="banniere">
    <h1>Bienvenue à Labo Academy</h1>
    <p>Apprenez le <strong>développement web</strong>, le <em>machine learning</em>
et le génie logiciel avec des projets concrets.</p>
    <a href="#inscription" class="bouton">Je m'inscris</a>
  </div>

  <main>

    <section id="formations" class="section-formations">
      <h2>Nos formations</h2>

      <article class="carte">
```

```
        <h3>Développement Web</h3>
        <p>HTML5, CSS3 et JavaScript pour créer des sites modernes.</p>
        <p>Durée : <span class="duree">24 heures</span></p>
      </article>

      <article class="carte">
        <h3>Machine Learning</h3>
        <p>Préparer des données, entraîner et évaluer un modèle.</p>
        <p>Durée : <span class="duree">30 heures</span></p>
      </article>

      <article class="carte">
        <h3>Génie Logiciel</h3>
        <p>UML, bonnes pratiques de code et travail en équipe.</p>
        <p>Durée : <span class="duree">36 heures</span></p>
      </article>
    </section>

    <section id="apropos" class="section-apropos">
      <h2>Pourquoi choisir Labo Academy ?</h2>
      <ul>
        <li>Des projets concrets à chaque cours</li>
        <li>Des enseignants en activité</li>
        <li>Un suivi personnalisé</li>
      </ul>

      <h3>Comment s’inscrire ?</h3>
      <ol>
        <li>Remplir le formulaire en ligne</li>
        <li>Valider son adresse e-mail</li>
        <li>Choisir sa filière</li>
      </ol>
    </section>

    <section id="inscription" class="section-inscription">
      <h2>Créer mon compte</h2>
      <!-- le formulaire arrive à l’étape 7 -->
    </section>

  </main>

  <footer class="pied" id="contact">
    <h2>Nous contacter</h2>
    <p>
      Labo Academy<br>
      Douala, Cameroun<br>
      <a href="mailto:contact@laboacademy.cm">contact@laboacademy.cm</a>
    </p>
    <p><small>© 2026 Labo Academy · EvangelisteDuCode</small></p>
  </footer>

</body>
```

Explications

- Le `header` contient **deux blocs** : un `div.logo` (image + nom, regroupés) et un `nav.menu`.

- La bannière est un `div` : c'est un bloc d'accroche sans équivalent sémantique précis.
- Chaque `section` a un `id` : les liens du menu (`href="#formations"`) fonctionnent maintenant. Cliquez-les pour tester.
- Chaque formation est un `article` : sa carte a un sens autonome.
- Les `class` (`carte`, `banniere`, `bouton`…) ne changent rien pour l'instant, mais elles préparent la séance de CSS3.
- Le lien `class="bouton"` est un lien (`a`) qui pointe vers l'ancre `#inscription`, pas un `button` : un `button` fait une action, un lien mène quelque part.

Étape 7 : Le formulaire d'inscription

Objectif : permettre à l'utilisateur de saisir des informations.

Les éléments d'un formulaire

Balise	Rôle
`<form>`	Enveloppe tout le formulaire. `action` indique où envoyer les données, `method` comment (`get` ou `post`).
`<label>`	Libellé d'un champ, relié par `for` à l'`id` du champ.
`<input>`	Champ de saisie ; son `type` change tout (`text`, `email`, `tel`, `date`, `password`, `radio`, `checkbox`, `submit`…).
`<select>` et `<option>`	Liste déroulante.
`<textarea>`	Zone de texte sur plusieurs lignes.
`<fieldset>` et `<legend>`	Regroupe des champs liés et lui donne un titre.
`<button>`	Bouton ; `type="submit"` envoie le formulaire.

Attributs importants

- `name` : nom de la donnée envoyée au serveur (et lue plus tard en JavaScript) ;
- `id` : identifiant unique, relié au `label` ;
- `placeholder` : exemple grisé dans le champ vide (**ne remplace jamais le `label`**) ;
- `required`, `minlength`, `maxlength`, `min`, `max`, `pattern` : contraintes natives, que nous exploiterons à la validation ;
- `value` : valeur envoyée (boutons radio, options) ou valeur initiale ;
- `checked`, `selected` : coche ou sélectionne par défaut.

Application au projet

Dans la `section#inscription`, remplacez le commentaire par :

```
<form id="form-inscription" action="#" method="post">

  <fieldset>
    <legend>Informations personnelles</legend>
```

```
    <div class="champ">
      <label for="nom">Nom complet</label>
      <input type="text" id="nom" name="nom" placeholder="Ex : Jean Mbarga" required>
    </div>

    <div class="champ">
      <label for="email">Adresse e-mail</label>
      <input type="email" id="email" name="email" placeholder="vous@exemple.com"
required>
    </div>

    <div class="champ">
      <label for="tel">Téléphone</label>
      <input type="tel" id="tel" name="tel" placeholder="6XXXXXXXX">
    </div>

    <div class="champ">
      <label for="naissance">Date de naissance</label>
      <input type="date" id="naissance" name="naissance">
    </div>

    <div class="champ">
      <label for="filiere">Filière souhaitée</label>
      <select id="filiere" name="filiere" required>
        <option value="">-- Choisir --</option>
        <option value="web">Développement Web</option>
        <option value="ml">Machine Learning</option>
        <option value="gl">Génie Logiciel</option>
      </select>
    </div>
  </fieldset>

  <fieldset>
    <legend>Sécurité du compte</legend>

    <div class="champ">
      <label for="mdp">Mot de passe</label>
      <input type="password" id="mdp" name="mdp" minlength="8" required>
    </div>

    <div class="champ">
      <label for="mdp2">Confirmer le mot de passe</label>
      <input type="password" id="mdp2" name="mdp2" required>
    </div>
  </fieldset>

  <fieldset>
    <legend>Votre niveau</legend>
    <label><input type="radio" name="niveau" value="debutant" checked>
Débutant</label>
    <label><input type="radio" name="niveau" value="intermediaire">
Intermédiaire</label>
    <label><input type="radio" name="niveau" value="avance"> Avancé</label>
  </fieldset>
```

```
  <div class="champ">
    <label for="motivation">Motivation (facultatif)</label>
    <textarea id="motivation" name="motivation" rows="4" placeholder="Pourquoi
souhaitez-vous suivre cette formation ?"></textarea>
  </div>

  <label class="conditions">
    <input type="checkbox" id="cgu" name="cgu" required>
    J’accepte les conditions d’utilisation
  </label>

  <button type="submit" class="bouton">S’inscrire</button>

</form>
```

Explications

- `label for="nom"` + `input id="nom"` : le lien entre les deux permet de cliquer sur le texte pour activer le champ, et aide les lecteurs d’écran.
- Le `type` apporte un comportement natif : `email` vérifie la forme de l’adresse, `date` affiche un calendrier, `tel` ouvre le pavé numérique sur mobile, `password` masque la saisie.
- Les trois boutons radio portent le **même `name`** (`niveau`) : c’est ce qui les rend exclusifs. Seul le `value` du bouton coché est envoyé.
- Chaque champ est enveloppé dans un `div.champ` : le regroupement (label + champ) facilitera la mise en page et, plus tard, l’affichage des messages d’erreur en JavaScript.
- Les attributs `required` et `minlength` sont de premières contraintes : essayez d’envoyer le formulaire vide, le navigateur réagit déjà. Nous irons beaucoup plus loin à la séance de validation.

Étape 8 : Vérification finale

Objectif : s’assurer que le code est propre avant de passer au CSS3.

Validation : copiez le contenu de `index.html` dans le validateur du W3C (validator.w3.org) et corrigez les erreurs signalées.

Checklist

- ☐`<!DOCTYPE html>`, `lang="fr"`, `charset`, `title` présents.
- ☐Un seul `h1` et des titres dans l’ordre (`h1`, `h2`, `h3`).
- ☐Toutes les images ont un `alt`.
- ☐Aucun `id` n’est utilisé deux fois.
- ☐Chaque champ de formulaire a un `label` relié (`for` / `id`).
- ☐Les liens du menu font défiler jusqu’à la bonne section.
- ☐Les balises sont bien fermées et correctement indentées.
- ☐Le projet est versionné : `git init`, `git add .`, `git commit -m "Étape HTML5 terminée"`.

Erreurs fréquentes

Erreur	Correction

Erreur	Correction
Plusieurs `h1`	un seul `h1`, puis des `h2`
`  ` pour espacer	utiliser des paragraphes, le CSS gérera les espaces
Image sans `alt`	toujours décrire l'image
`label` non relié au champ	`for` doit être identique à l'`id`
`div` partout	utiliser `header`, `nav`, `section`, `article`, `footer`
Balises mal imbriquées	fermer dans l'ordre inverse de l'ouverture
Même `id` répété	`id` unique ; utiliser `class` pour répéter

Fiche récapitulative

Famille	Balises à retenir
Squelette	`<!DOCTYPE html>`, `html`, `head`, `meta`, `title`, `body`
Texte	`h1` à `h6`, `p`, `br`, `hr`, `strong`, `em`, `small`
Listes	`ul`, `ol`, `li`
Liens et médias	`a` (`href`, `target`), `img` (`src`, `alt`)
Conteneurs	`div` (bloc), `span` (en ligne), attributs `class` et `id`
Structure	`header`, `nav`, `main`, `section`, `article`, `aside`, `footer`
Formulaire	`form`, `label`, `input`, `select`, `option`, `textarea`, `fieldset`, `legend`, `button`

Exercices d'application

1. **Page « À propos »** : créez `apropos.html` avec le même en-tête et le même pied de page, puis reliez-la au menu.
2. **Nouvelle formation** : ajoutez un quatrième `article` « Cybersécurité » dans la section des formations.
3. **Liste imbriquée** : dans la section À propos, présentez les modules du cours de développement web en liste imbriquée (HTML, CSS, JavaScript et leurs sous-parties).
4. **Nouveau champ** : ajoutez au formulaire un champ « Ville » sous forme de `select` de cinq villes, avec son `label`.
5. **Tableau des tarifs** : ajoutez un tableau (`table`, `thead`, `tbody`, `tr`, `th`, `td`) récapitulant les formations, leurs durées et leurs prix.
6. **Diagnostic** : trouvez et corrigez les trois erreurs de ce code.

```
<h1>Labo Academy</h1>
<h1>Nos formations</h1>
```

```
<img src="img/logo.png">
<p><strong>Inscription gratuite</p></strong>
<label>Nom</label>
<input type="text" id="nom">
```

Partie B : CSS3

La page est structurée mais brute. Nous la mettons maintenant en forme avec **CSS** (*Cascading Style Sheets*), en nous appuyant sur les `class` et les `id` posés dans la partie A. C'est la raison pour laquelle nous les avons nommés avec soin.

Étape	Notion	Application dans le projet
9	La syntaxe CSS	Brancher `style.css`, sélecteurs, cascade, box model
10	Mise en forme de base	Couleurs, typographie, cartes, formulaire, boutons
11	Flexbox	En-tête, menu et champs du formulaire
12	Grid	Cartes de formations et formulaire sur deux colonnes
13	Responsive	Adapter la page aux tablettes et téléphones
14	Vérifications finales	Checklist, fiche récapitulative, exercices

Durée conseillée : 3 h (séance 2 du cours).

Étape 9 : La syntaxe CSS

Objectif : comprendre comment écrire une règle CSS et la brancher à la page.

Relier le CSS à la page

Méthode	Exemple	Usage
Fichier externe	`<link rel="stylesheet" href="css/style.css">` dans le `head`	**Recommandée** : un seul fichier pour toutes les pages
Balise `style`	`<style> h1 { color: red; } </style>` dans le `head`	Tests rapides
Attribut `style`	`<p style="color: red;">`	À éviter : mélange contenu et présentation

Application : créez le fichier `css/style.css` (vide pour l'instant), puis, dans le `head` de `index.html`, après la balise `title`, ajoutez :

```
<link rel="stylesheet" href="css/style.css">
```

Anatomie d'une règle

```
selecteur {
  propriete: valeur;
```

```
  autre-propriete: valeur;
}
```
Exemple :

```
h1 {
  color: #1d4ed8;   /* couleur du texte */
  font-size: 2rem;  /* taille du texte */
}
```

- Le **sélecteur** désigne les éléments à styliser (ici tous les `h1`).
- Les **déclarations** sont entre accolades `{ }` : chacune associe une **propriété** à une **valeur**, séparées par `:` et terminées par `;`.
- Les commentaires s'écrivent `/* … */`.
- Une erreur de syntaxe (point-virgule oublié) peut faire ignorer la déclaration suivante : vérifiez toujours les `;`.

Les sélecteurs

Sélecteur	Syntaxe	Ce qu'il cible	Exemple dans le projet
Élément	`p`	toutes les balises `p`	`p { margin-bottom: 1rem; }`
Classe	`.carte`	tous les éléments `class="carte"`	`.carte { padding: 1rem; }`
Identifiant	`#inscription`	l'unique élément `id="inscription"`	`#inscription { scroll-margin-top: 1rem; }`
Groupe	`h1, h2, h3`	plusieurs sélecteurs à la fois	`h1, h2 { font-weight: 700; }`
Descendant	`.menu a`	les `a` situés dans `.menu`	`.menu a:hover { … }`
Enfant direct	`.menu > ul`	les `ul` enfants directs de `.menu`	`.menu > ul { … }`
Pseudo-classe	`a:hover`	un élément dans un certain état (`:hover`, `:focus`)	`.bouton:hover { … }`
Attribut	`input[type="email"]`	les éléments ayant cet attribut	`input[type="email"] { … }`
Universel	`*`	tous les éléments	`* { box-sizing: border-box; }`

Unités et couleurs

- **Unités de longueur** : `px` (fixe), `rem` (relative à la taille de texte de la page, **à privilégier**), `em`, `%` (relative au parent), `vw` / `vh` (relatives à la fenêtre).

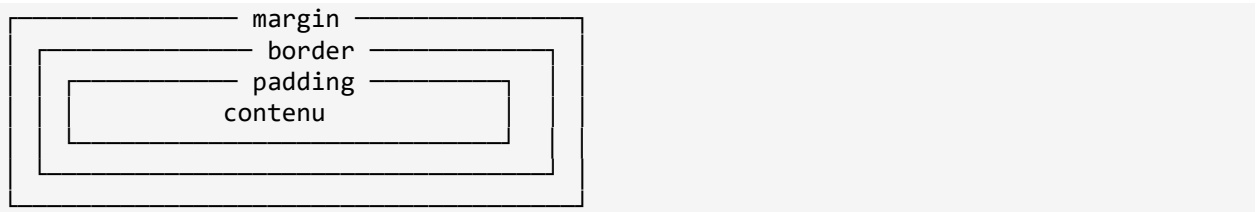
- **Couleurs** : nom (`red`), hexadécimal (`#1d4ed8`), `rgb(29, 78, 216)`, `rgba(0, 0, 0, 0.5)` (avec transparence).

Cascade, spécificité et héritage

- Quand deux règles visent le même élément, la plus **spécifique** gagne : `id` > `class` > élément. À spécificité égale, la **dernière** écrite l'emporte.
- Certaines propriétés sont **héritées** par les enfants (`color`, `font-family`, `line-height`) ; d'autres non (`margin`, `border`).
- Évitez `!important` : il casse la logique de la cascade.

Le modèle de boîte (box model)

Tout élément HTML est une boîte composée, de l'intérieur vers l'extérieur, du contenu, du `padding`, de la bordure (`border`) et de la marge (`margin`).



- `padding` : espace **intérieur**, entre le contenu et la bordure.
- `margin` : espace **extérieur**, entre cette boîte et ses voisines.
- `box-sizing: border-box` : `width` inclut le `padding` et la bordure ; les calculs de largeur deviennent prévisibles. On l'applique à tous les éléments.
- `margin: 0 auto` centre horizontalement un bloc dont la largeur est limitée (`max-width`).

Étape 10 : Mise en forme de base du projet

Objectif : donner une identité visuelle à la page avec les propriétés essentielles (couleurs, texte, boîtes). La mise en page viendra avec Flexbox et Grid.

Dans `css/style.css`, écrivez les blocs suivants **dans cet ordre**.

1. Variables et réinitialisation

```
/* ===== Variables ===== */
:root {
  --couleur-principale: #1d4ed8;
  --couleur-accent: #f59e0b;
  --couleur-fond: #f3f4f6;
  --couleur-texte: #1f2937;
  --couleur-erreur: #dc2626;
  --couleur-succes: #16a34a;
  --rayon: 8px;
}

/* ===== Réinitialisation ===== */
*,
*::before,
```

```
*::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

- Les **variables CSS** (`--nom`), déclarées dans `:root`, centralisent les valeurs répétées. On les utilise avec `var(--nom)`. Pour changer de thème, on modifie une seule ligne.
- Le **reset** supprime les marges et paddings par défaut, qui varient d'un navigateur à l'autre : on repart d'une base identique partout.

2. Typographie et éléments de base

```
/* ===== Base ===== */
body {
  font-family: system-ui, Arial, sans-serif;
  line-height: 1.6;
  color: var(--couleur-texte);
  background: var(--couleur-fond);
}

h1 { font-size: 2.25rem; }
h2 { font-size: 1.6rem; margin-bottom: 1rem; }
h3 { font-size: 1.2rem; margin-bottom: 0.5rem; }

p, ul, ol { margin-bottom: 1rem; }
ul, ol { padding-left: 1.5rem; }

a { color: inherit; text-decoration: none; }
img { max-width: 100%; height: auto; }
```

- `font-family` : liste de polices de repli ; le navigateur prend la première disponible.
- `line-height: 1.6` aère le texte et améliore la lecture.
- `max-width: 100%` sur les images évite qu'elles dépassent de leur conteneur.

3. En-tête, bannière et pied de page

```
/* ===== En-tête ===== */
.entete {
  background: var(--couleur-principale);
  color: #fff;
  padding: 1rem 2rem;
}

.logo { font-weight: 700; font-size: 1.25rem; }
.menu a:hover { color: var(--couleur-accent); }

/* ===== Bannière ===== */
.banniere {
  background: linear-gradient(135deg, var(--couleur-principale), #1e3a8a);
  color: #fff;
  text-align: center;
  padding: 4rem 1rem;
}

.banniere p {
```

```
  max-width: 600px;
  margin: 1rem auto 1.5rem;
}

/* ===== Pied de page ===== */
.pied {
  background: var(--couleur-texte);
  color: #fff;
  text-align: center;
  padding: 2rem 1rem;
}
```

- `.menu a:hover` : sélecteur descendant + pseudo-classe ; la couleur change au survol.
- `margin: 1rem auto 1.5rem` : marge haute 1rem, côtés `auto` (centrage), marge basse 1.5rem.
- `linear-gradient(…)` crée un dégradé, utilisable comme `background`.

4. Sections, cartes et boutons

```
/* ===== Sections et cartes ===== */
main section {
  max-width: 1000px;
  margin: 0 auto;
  padding: 2rem 1rem;
}

.carte {
  background: #fff;
  padding: 1.25rem;
  border-radius: var(--rayon);
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.duree { font-weight: 600; color: var(--couleur-principale); }

/* ===== Boutons ===== */
.bouton {
  display: inline-block;
  padding: 0.75rem 1.5rem;
  border: none;
  border-radius: var(--rayon);
  background: var(--couleur-accent);
  color: var(--couleur-texte);
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.2s;
}

.bouton:hover { background: #d97706; }
```

- `display: inline-block` permet à un lien (`a`) d'avoir un `padding` complet comme un bouton.
- `border-radius` arrondit les coins ; `box-shadow` ajoute une ombre légère (décalage x, y, flou, couleur).
- `transition` adoucit le changement de couleur au survol.

- Ici la classe `.bouton` sert à la fois au lien de la bannière et au bouton du formulaire : **une classe, plusieurs éléments**.

5. Formulaire

```
/* ===== Formulaire ===== */
fieldset {
  border: 1px solid #d1d5db;
  border-radius: var(--rayon);
  padding: 1rem;
  margin-bottom: 1.25rem;
}

legend {
  padding: 0 0.5rem;
  font-weight: 600;
  color: var(--couleur-principale);
}

label { font-weight: 500; }

input[type="text"],
input[type="email"],
input[type="tel"],
input[type="date"],
input[type="password"],
select,
textarea {
  width: 100%;
  padding: 0.6rem 0.75rem;
  border: 1px solid #9ca3af;
  border-radius: var(--rayon);
  font: inherit;
}

input:focus,
select:focus,
textarea:focus {
  outline: none;
  border-color: var(--couleur-principale);
  box-shadow: 0 0 0 3px rgba(29, 78, 216, 0.25);
}

.conditions { display: block; margin: 1rem 0; }

/* États préparés pour la validation en JavaScript */
.champ.error input,
.champ.error select { border-color: var(--couleur-erreur); }

.champ.success input,
.champ.success select { border-color: var(--couleur-succes); }

.message-erreur {
  color: var(--couleur-erreur);
```

```
  font-size: 0.875rem;
}
```

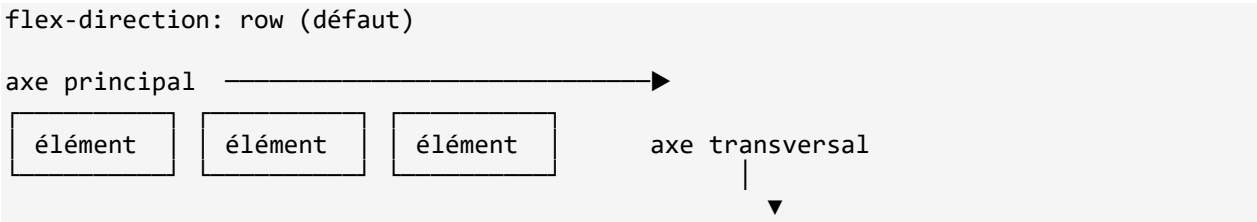
- Les **sélecteurs d'attribut** (`input[type="email"]`) évitent de styliser les cases à cocher et boutons radio comme des champs texte.
- `font: inherit` fait hériter les champs de la police de la page (par défaut, ils en ont une propre).
- `:focus` montre quel champ est actif ; ne supprimez jamais l'indicateur de focus sans le remplacer.
- Les classes `.error` et `.success` **ne s'affichent pas encore** : en JavaScript, nous les ajouterons avec `classList.add()` pendant la validation. On prépare le terrain.
- Évitez `:invalid` pour colorer les erreurs : les champs obligatoires vides s'affichent en rouge dès le chargement de la page.

Étape 11 : Flexbox, aligner sur une ligne ou une colonne

Objectif : organiser des éléments **sur un seul axe** (horizontal ou vertical) : menu, en-tête, label + champ.

Le principe

On déclare `display: flex` sur le **conteneur** ; ses **enfants directs** deviennent des éléments flex et se rangent côte à côte.



- L'**axe principal** suit `flex-direction` (horizontal en `row`, vertical en `column`).
- L'**axe transversal** est perpendiculaire.

Propriétés du conteneur

Propriété	Rôle	Valeurs courantes
`display`	active Flexbox	`flex`
`flex-direction`	sens de l'axe principal	`row`, `column`
`justify-content`	répartition sur l'**axe principal**	`flex-start`, `center`, `flex-end`, `space-between`, `space-around`, `space-evenly`
`align-items`	alignement sur l'**axe transversal**	`stretch` (défaut), `center`, `flex-start`, `flex-end`
`flex-wrap`	passage à la ligne si manque de place	`nowrap` (défaut), `wrap`
`gap`	espace entre les éléments	`1rem`, `1rem 2rem`

Propriétés des éléments

Propriété	Rôle	Exemple
`flex: 1`	l'élément se partage l'espace disponible	`.champ-large { flex: 1; }`
`align-self`	alignement d'un seul élément	`align-self: flex-end;`
`order`	change l'ordre d'affichage	`order: -1;`

Application au projet

Ajoutez à la suite de `style.css` :

```
/* ===== Flexbox ===== */

/* En-tête : logo à gauche, menu à droite, centrés verticalement */
.entete {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

/* Logo : image et nom côte à côte */
.logo {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

/* Menu : la liste devient horizontale */
.menu ul {
  display: flex;
  gap: 1.5rem;
  list-style: none;
  padding-left: 0;
  margin-bottom: 0;
}

/* Chaque champ : label au-dessus, champ en dessous */
.champ {
  display: flex;
  flex-direction: column;
  margin-bottom: 1rem;
}

.champ label { margin-bottom: 0.25rem; }

/* Boutons radio : alignés sur une ligne */
fieldset .radio-groupe {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}
```

Pour la dernière règle, entourez les trois boutons radio dans `index.html` d'un `div` :

```
<fieldset>
  <legend>Votre niveau</legend>
  <div class="radio-groupe">
    <label><input type="radio" name="niveau" value="debutant" checked>
Débutant</label>
    <label><input type="radio" name="niveau" value="intermediaire">
Intermédiaire</label>
    <label><input type="radio" name="niveau" value="avance"> Avancé</label>
  </div>
</fieldset>
```

Explications

- `justify-content: space-between` pousse le premier élément à gauche et le dernier à droite, avec l'espace au milieu : c'est la recette classique de l'en-tête.
- `align-items: center` centre verticalement : plus besoin de bidouiller avec des marges.
- `gap` remplace les marges entre éléments : l'espace n'apparaît qu'**entre** eux, jamais aux extrémités.
- `display: flex` sur `.menu ul` rend les `li` horizontaux : le menu est une **liste**, mais s'affiche en ligne.
- `flex-direction: column` empile le `label` et le champ.
- Le `div.radio-groupe` illustre le rôle de `div` : il regroupe des éléments **uniquement pour la mise en page**.

À retenir : Flexbox s'applique au **parent**. Si rien ne bouge, vérifiez que vous avez bien écrit `display: flex` sur le conteneur, pas sur les enfants.

Étape 12 : Grid, organiser en lignes et en colonnes

Objectif : construire des mises en page **en deux dimensions** (lignes et colonnes) : galerie de cartes, formulaire sur deux colonnes, page entière.

Le principe

On déclare `display: grid` sur le **conteneur**, puis on définit les colonnes avec `grid-template-columns`. Les enfants directs se placent automatiquement dans les cases.

```
.conteneur {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;  /* trois colonnes égales */
  gap: 1rem;                            /* espace entre les cases */
}
```

- `fr` (*fraction*) : une part de l'espace disponible. `1fr 2fr` donne une colonne deux fois plus large que l'autre.
- On peut mélanger les unités : `250px 1fr` = une colonne fixe, une colonne qui prend le reste.

Propriétés essentielles

Propriété	Rôle	Exemple

Propriété	Rôle	Exemple
`display: grid`	active Grid sur le conteneur	
`grid-template-columns`	définit les colonnes	`1fr 1fr`, `repeat(3, 1fr)`, `250px 1fr`
`gap`	espace entre lignes et colonnes	`1rem`, `1rem 2rem`
`grid-column`	place un élément sur plusieurs colonnes	`grid-column: 1 / -1;` (toute la largeur), `grid-column: span 2;`
`grid-template-areas`	dessine la page avec des noms de zones	voir plus bas
`repeat(auto-fit, minmax(250px, 1fr))`	autant de colonnes que la place le permet	cartes adaptables sans media query

Application au projet

1. Les cartes de formations. Ajoutez à `style.css` :

```
/* ===== Grid ===== */

/* Cartes : autant de colonnes de 250px minimum que possible */
.section-formations {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 1.5rem;
}

/* Le titre h2 occupe toute la largeur */
.section-formations h2 {
  grid-column: 1 / -1;
}
```

- Le `h2` est lui aussi un enfant direct de la section : sans règle, il occuperait la première case. `grid-column: 1 / -1` l'étend de la première à la dernière colonne.
- `repeat(auto-fit, minmax(250px, 1fr))` : chaque carte fait au moins 250px ; le navigateur en place autant qu'il peut par ligne. C'est **déjà responsive**, sans media query.

2. Le formulaire sur deux colonnes. Ajoutez la classe `grille-form` aux deux premiers `fieldset` de `index.html` (informations personnelles et sécurité) :

```
<fieldset class="grille-form">
  <legend>Informations personnelles</legend>
  …
</fieldset>
```

Puis dans le CSS :

```
.grille-form {
  display: grid;
  grid-template-columns: 1fr 1fr;
```

```
  gap: 0 1rem;
}

/* Un champ qui occupe toute la largeur */
.champ-large {
  grid-column: 1 / -1;
}
```

- On cible **uniquement** les `fieldset` qui portent `grille-form` : le fieldset des boutons radio garde sa mise en page Flexbox. C'est l'intérêt de `class` : choisir précisément ce qu'on stylise.
- Pour qu'un champ (par exemple l'e-mail) prenne toute la largeur, ajoutez `champ-large` à son `div` : `<div class="champ champ-large">`. Un élément peut avoir plusieurs classes séparées par un espace.

Pour aller plus loin : reproduire la Figure 2 avec `grid-template-areas`

On peut dessiner la mise en page comme un plan. Enveloppez le contenu dans un `div class="page"` et donnez un nom à chaque zone :

```
.page {
  display: grid;
  grid-template-columns: 250px 1fr;
  grid-template-areas:
    "entete entete"
    "cote   principal"
    "pied   pied";
  gap: 1rem;
}

.entete { grid-area: entete; }
aside   { grid-area: cote; }
main    { grid-area: principal; }
.pied   { grid-area: pied; }
```

Le code se lit comme le schéma de la Figure 2 : en-tête sur toute la largeur, barre latérale à gauche, contenu à droite, pied de page en bas.

Flexbox ou Grid ?

	Flexbox	Grid
Dimension	**une** (ligne **ou** colonne)	**deux** (lignes **et** colonnes)
Logique	le contenu dicte la mise en page	la grille dicte la mise en page
Usage typique	menu, en-tête, label + champ, boutons alignés	galerie de cartes, formulaire en colonnes, page entière

Les deux se combinent : dans notre projet, Grid organise les cartes et Flexbox aligne leur contenu ou le menu.

Étape 13 : Un peu de responsive

Objectif : adapter la page à toutes les tailles d'écran (ordinateur, tablette, téléphone).

Les bases du responsive

1. **La balise `viewport`** (déjà dans le `head`, étape 1) : sans elle, les téléphones affichent la page comme sur un grand écran, en réduit.
2. **Des unités souples** : `%`, `rem`, `max-width` plutôt que des largeurs fixes en `px`.
3. **Des images flexibles** : `img { max-width: 100%; height: auto; }` (déjà fait à l'étape 10).
4. **Des mises en page qui s'adaptent seules** : `flex-wrap: wrap` ou `repeat(auto-fit, minmax(…))` (déjà fait pour les cartes).
5. **Les media queries** : des règles qui ne s'appliquent que sous certaines conditions.

La syntaxe d'une media query

```
@media (max-width: 768px) {
  /* règles appliquées quand l'écran fait 768px de large ou moins */
  .menu ul { flex-direction: column; }
}
```

- `max-width: 768px` : « jusqu'à 768 px de large » (approche *desktop-first*).
- `min-width: 768px` : « à partir de 768 px » (approche *mobile-first* : on écrit d'abord le style mobile, puis on enrichit pour les grands écrans).

Appareil	Largeur approximative	Point de rupture courant
Téléphone	moins de 600 px	`max-width: 600px`
Tablette	600 à 1024 px	`max-width: 768px`
Ordinateur	plus de 1024 px	style par défaut

Les points de rupture ne sont pas imposés : on les place **là où la mise en page casse**, pas selon un appareil précis.

Application au projet

Ajoutez à la **fin** de `style.css` (les media queries doivent venir en dernier pour l'emporter sur les règles précédentes) :

```
/* ===== Responsive ===== */

@media (max-width: 768px) {
  /* En-tête et menu empilés */
  .entete {
    flex-direction: column;
    gap: 0.75rem;
    padding: 1rem;
  }

  .menu ul {
    flex-wrap: wrap;
    justify-content: center;
    gap: 1rem;
  }

  /* Formulaire sur une seule colonne */
```

```
  .grille-form {
    grid-template-columns: 1fr;
  }

  /* Bannière plus compacte */
  .banniere { padding: 2.5rem 1rem; }
  h1 { font-size: 1.75rem; }
}

@media (max-width: 480px) {
  .bouton {
    display: block;
    width: 100%;
    text-align: center;
  }
}
```

Explications

- Sous 768 px, l'en-tête passe en **colonne** : logo au-dessus, menu en dessous (`flex-direction: column`).
- Le menu peut retourner à la ligne (`flex-wrap: wrap`) s'il manque de place.
- Le formulaire passe de deux colonnes à une (`1fr`) : il reste lisible sur téléphone.
- Sous 480 px, les boutons occupent toute la largeur : plus faciles à toucher du doigt.
- Les cartes de formations **n'ont besoin d'aucune media query** : `auto-fit` ajuste le nombre de colonnes.

Tester : dans le navigateur, ouvrez les outils de développement (touche F12), activez le **mode appareil** (icône téléphone/tablette) et choisissez un appareil ou redimensionnez la fenêtre. Vérifiez qu'aucun défilement horizontal n'apparaît.

Étape 14 : Vérifications finales du CSS

Checklist

- ☐`style.css` est bien lié dans le `head` (chemin `css/style.css` correct).
- ☐Les couleurs viennent des variables `var(--…)`, pas de valeurs répétées.
- ☐Le menu est horizontal sur ordinateur et s'empile sur mobile.
- ☐Les cartes de formations se répartissent en colonnes et passent à la ligne seules.
- ☐Le formulaire est sur deux colonnes sur ordinateur et sur une colonne sur mobile.
- ☐Le focus est visible quand on navigue avec la touche Tab.
- ☐Aucun défilement horizontal à 360 px de large.
- ☐Le projet est versionné : `git add .` puis `git commit -m "Étape CSS3 terminée"`.

Erreurs fréquentes

Erreur	Cause et correction
La règle n'a aucun effet	vérifier l'orthographe du sélecteur (`.carte` avec le point), le `;` précédent, le chemin du fichier CSS

Erreur	Cause et correction
`display: flex` ne change rien	il doit être sur le **parent**, pas sur les enfants
Un style est écrasé	une règle plus spécifique ou placée plus bas gagne : inspecter l'élément dans les DevTools
La page dépasse à droite sur mobile	largeur fixe en `px` ou image sans `max-width: 100%`
Media query sans effet	placée avant la règle qu'elle doit remplacer, ou balise `viewport` absente
Marges inattendues	oublier `box-sizing: border-box` ou le reset

Fiche récapitulative CSS

Notion	À retenir
Syntaxe	`selecteur { propriete: valeur; }`
Sélecteurs	`p` (élément), `.classe`, `#id`, `a:hover`, `.menu a`, `input[type="email"]`
Cascade	`id` > `class` > élément ; à égalité, la dernière règle gagne
Box model	contenu, `padding`, `border`, `margin` ; `box-sizing: border-box`
Variables	`--nom` dans `:root`, utilisée avec `var(--nom)`
Flexbox	sur le parent : `display: flex`, `flex-direction`, `justify-content`, `align-items`, `gap`, `flex-wrap`
Grid	sur le parent : `display: grid`, `grid-template-columns`, `gap` ; `grid-column` sur l'enfant
Responsive	`viewport`, unités souples, `@media (max-width: …)`
États	`:hover`, `:focus` ; classes `.error` / `.success` pour la validation

Exercices d'application

1. **Thème** : changez toute la palette du site en modifiant uniquement les variables de `:root`. Que constatez-vous ?
2. **Carte** : ajoutez un effet au survol des cartes (ombre plus forte, léger déplacement vers le haut avec `transform: translateY(-4px)` et `transition`).
3. **Flexbox** : centrez le texte et le bouton de la bannière en colonne, à la fois horizontalement et verticalement, avec `display: flex`, `flex-direction: column` et `align-items: center`.
4. **Grid** : faites occuper toute la largeur au champ « Adresse e-mail » avec la classe `champ-large`.
5. **Responsive** : ajoutez une media query à 1024 px qui réduit le `padding` des sections.
6. **Pied de page** : avec Flexbox, placez l'adresse et les mentions légales côte à côte sur ordinateur et l'une sous l'autre sur mobile.

7. **Défi** : reproduisez la mise en page de la Figure 2 (en-tête, barre latérale, contenu, pied de page) avec `grid-template-areas`.

Pour la suite

La page est structurée (HTML5) et mise en forme (CSS3). Aux séances suivantes, nous lui donnons vie avec **JavaScript** : lire les champs avec le DOM, réagir aux événements et valider le formulaire en ajoutant les classes `.error` et `.success` que nous avons préparées à l'étape 10.